Consolidation: lessons 1 to 3, one call at a time

Technologies for Artificial Intelligence — self-study, and two hours in class

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable

Contents

What this is . . . 1
How to use an atom . . . 1
The files . . . 2
The atoms . . . 2
00 — the scikit-learn API . . . 2
01 — lesson 1 . . . 3
02 — lesson 2 . . . 3
03 — lesson 3 . . . 4
Two hours in class . . . 4
What students will get wrong, and why it is reasonable . . . 5

What this is

Lessons 1 to 3 ran every scikit-learn call on hundreds of rows: 569 tumours, 2,000 customers, 600 houses. At that size the result of a call is visible, and what the call *did* is not. This folder takes the same calls one at a time, on two to ten rows, small enough that every number can be checked with a pencil.

Each section of a notebook is an **atom**: one idea, one call. Many atoms recompute the worked examples of the lesson 2 and lesson 3 commentaries (`Docs/*_commentary.md`) with the library, so that the arithmetic done by hand and the call that does it sit side by side; the numbers are the same.

It is **not examinable**, and it adds no new material. It does not replace the lesson notebooks either: every atom names the slide and the notebook section where its idea is treated at full scale.

Reading time: about 40 minutes per notebook, so two and a half to three hours for the four, done properly.

How to use an atom

Every atom has the same shape:

1. **What it shows**, in a sentence or two, and *From:* the lesson slide (by title) and notebook section it comes from.
2. **The data**, tiny, printed.

3. **Your prediction, before you run the next cell**: one concrete question.
4. **The code**: a few lines, with the call being taught in plain sight.
5. **What to look at**: what the output means, in the units of the problem.
6. **Try changing**: one small change to make and run.

The third step is the one that matters. Write the answer down, even as a single number, *before* running. A prediction that turns out wrong marks the exact place where your picture of the library differs from what it does, and that is worth more than any number of predictions that came out right.

The files

File	What it covers	Atoms
`00_the_scikit_learn_api.ipynb`	`fit`, `transform`, `fit_transform`, `predict`, `predict_proba`, `score`, the shape of `X`, `random_state`, `Pipeline`	8
`01_lesson_1_atoms.ipynb`	splitting, baselines, accuracy, the confusion matrix, precision and recall, two ways a model misleads	9
`02_lesson_2_atoms.ipynb`	looking, missing values, outliers, scaling, encoding, `ColumnTransformer`, the area under the curve, two leaks	17
`03_lesson_3_atoms.ipynb`	cost, R^2, the normal equation, gradient descent, Ridge and Lasso, what a coefficient means	18

They run in the course container like every other notebook, each in a few seconds, with no network. Start with `00`: the other three assume it.

The atoms

Italic titles are worked examples of the lesson's commentary: same data, same numbers.

00 — the scikit-learn API

#	Atom	Lesson source
1	Estimators learn in `fit`, and keep what they learned	L1 *Step 5: pipeline, not two steps*; L2 *What to do about a gap*
2	`transform` applies what `fit` learned; `fit_transform` does both	L2 *StandardScaler vs MinMaxScaler*
3	`predict` gives a label, `predict_proba` the probabilities behind it	L1 *Behind every label, a probability*
4	`score` is a metric with a default	L1 *Accuracy: the share of right answers*; L3 *A cost needs something to be compared against*
5	Why arrays, and why `X` has two dimensions	L1 notebook 2; L3 *θ and X: all parameters, all houses*

#	Atom	Lesson source
6	`random_state`: the same randomness, every time	L1 *One split is one measurement*
7	`Pipeline`: one `fit`, and every step fitted on the right rows	L1 *Step 5*; L2 *ColumnTransformer + Pipeline*
8	The contract, in one table	all three lessons

01 — lesson 1

#	Atom	Lesson source
1	`train_test_split`, and what `stratify` keeps	*Step 3: split, before anything else*
2	`DummyClassifier`: the score to beat, and two ways to read it	*Step 4: baseline first*
3	Behind every label, a probability, and a threshold nobody chose	*Behind every label, a probability*; *The trade-off, drawn*
4	The confusion matrix: where the right and wrong answers fall	*What accuracy hides*
5	Precision and recall, counted and then printed	*Precision and recall, counted*
6	A first real model, small enough to see	*Step 6: the result*
7	An unbalanced dataset: high accuracy, nothing detected	*99% accuracy, detecting nothing*
8	Leakage: choosing features with the test rows	*Select features first, split second*; *The rule*
9	One split is one measurement	*One split is one measurement*

02 — lesson 2

#	Atom	Lesson source
1	Load and look	*Look before you touch anything*
2	Finding the missing values	*Where the gaps are*
3	Dropping rows and columns	*What to do about a gap*
4	Mean imputation: *four ages out of ten*	*What mean imputation actually costs*
5	The missing indicator: keeping the fact of the gap	*What to do about a gap*; *Choosing among the four*
6	Outliers: *the ruler made of the outlier*, and *the same eight values*	*Rule 1*; *Rule 2: interquartile range (IQR)*
7	Two scalers: *five customers, one outlier*	*StandardScaler vs MinMaxScaler*
8	Correlations with `corr`	notebook 1, *Correlations*
9	Standardisation by hand, and the test set	*Why scale?*
10	*One step size for two directions*	*A 100:1 variance ratio, and what it costs*
11	*Ten customers, three encodings*, and *the steps ordinal encoding assumes equal*	*One customer, three encodings*; *Ordinal: equal steps, unequal churn*
12	*Four rows, four columns, rank three*	*One one-hot column too many*; *4 columns, but only rank 3*
13	`ColumnTransformer`: a different preparation for each column	*ColumnTransformer + Pipeline*; *The code*

#	Atom	Lesson source
14	*The AUC by counting pairs*	*The model, on churn*
15	Leak 1: *the two nearest neighbours are both in the test set*	*Leak 1: impute before splitting*
16	Leak 2: *a zip code with no signal, and an AUC of 0.875*	*Leak 2: encode before splitting*; *How much of its own label a row sees*
17	The fix: *the same eight customers, cross-fitted*, and `TargetEncoder`	*The fix*

03 — lesson 3

#	Atom	Lesson source
1	*The best single price for three houses*	*Why squared, not absolute*
2	The cost of one guess	*Trying $\hat{y}$ = 2,400 · area + 45,000*
3	R^2, by hand and with `r2_score`	*A cost needs something to be compared against*
4	*The same model, two test sets*	*A cost needs something to be compared against*
5	The design matrix, the label vector, and the normal equation by hand	*θ and X*; *The same equation, by hand*
6	*One slope, no intercept*: the cost is a bowl	*The cost is a bowl*
7	*Two standardised features, rising correlation*	*How stretched is the valley?*
8	*Three houses voting on the price of a square metre*	*The gradients: one for w, one for b*
9	Gradient descent, one iteration at a time	*One iteration, then the next*; *One step on two houses*
10	*A bowl of curvature 2*, and four learning rates	*Three learning rates*
11	*A parabola by the normal equation*	*"Linear" means linear in the coefficients*
12	What a penalty buys	*What a penalty buys*
13	*One coefficient, four penalties*	*Ridge has an exact solution too*
14	*The same column twice*: *one effect, two columns*	*The case Ridge was invented for*
15	*Three coefficients, both penalties*	*Ridge shrinks, Lasso selects*
16	The coefficients are noise	*The coefficients are noise*
17	*The same column, two coefficients*	*What a coefficient actually says*
18	When can you trust a coefficient?	*When can you trust a coefficient?*

Two hours in class

A selection of thirteen atoms, run together from the projector. For each one: read the question aloud, give the room a minute to commit to an answer (a show of hands works for most), run the cell, then ask who was surprised and why. The surprise is the point; the explanation follows from it.

Minutes	Notebook · atom	What it settles
5	—	how an atom works: predict, then run
8	00 · 1	what `fit` learns and where it keeps it
10	00 · 2	`transform` against `fit_transform` on the test set
7	00 · 7	a pipeline fits each step on the training rows
8	01 · 1	what `stratify` keeps, on eight patients
12	01 · 4 and 5	the confusion matrix, then precision and recall from it
10	01 · 8	leakage on pure noise, and over twenty datasets
10	—	break
10	02 · 11	three encodings of ten customers
7	02 · 14	the area under the curve as a count of pairs
12	02 · 16 and 17	the target-encoding leak, and the fix
8	03 · 5	the normal equation on three houses
10	03 · 9	gradient descent, one iteration at a time
3	—	close: which prediction surprised you most?
120		

Everything not in the selection is left for self-study, and the guide's index above is where to find it.

What students will get wrong, and why it is reasonable

- **"`fit_transform` on the test set is the same thing, just shorter."** The name invites it: if `fit_transform` is `fit` plus `transform`, using it everywhere looks like tidiness. On the training set it is; on the test set it learns from the rows being measured. Atom 00 · 2 shows a test customer at 2.24 standard deviations above average becoming exactly average.
- **"`predict` is the model's answer."** It is the model's probability compared with 0.5, a threshold nobody chose for the problem at hand, and it is natural to take a default for a decision. Atom 01 · 3 shows a malignant tumour at probability 0.21 that the default never flags, and what a threshold of 0.2 costs to catch it.
- **"A higher R^2 means a better model."** Only on the same test set. Atom 03 · 4 shows the model's error falling by more than two thirds while R^2 falls from 0.971 to 0.625, because the second test set has less spread to explain.
- **"`TargetEncoder` is target encoding, so it leaks."** It is the one class in scikit-learn where `fit_transform` is not `fit` followed by `transform`: it cross-fits, and atom 02 · 17 shows the two giving different columns. The leak comes back only if `transform` is called on the training rows by hand.